Chapter 3: Software Frameworks for Real-Time and Embedded Systems

- Real-Time operating system (RTOS): definitions, Characteristics, function, Component and support for applications
- Inter process communication
- Real-Time task scheduling
- Scheduling
  - Cyclic scheduling
  - Priority-based scheduling
- Multi-tasking and Concurrency issue
- Handling resource sharing and dependencies
  - Priority inversion
  - Resource sharing protocols
- Fault- tolerance
- Synchronization techniques
  - Centralized clock synchronization algorithm
  - Distributed clock synchronization algorithm
- RTOS support for semaphores, queues, and events

What is a Software Framework in Embedded Systems?

- The **software framework of embedded systems** describes the **structure and organization of software components** that control the hardware and perform the required tasks in an embedded device.
- In the field of Embedded Systems, the framework helps developers to design **reliable, real-time, and efficient software** for devices such as those using microcontrollers like ARM Cortex-M0+ or ATmega32.
- A **software framework** is a **layered structure of software modules** that provide standard functions, libraries, and services for embedded applications.
- It defines: How software interacts with **hardware,** How **tasks and processes are managed** and How the **application logic** is implemented
- The framework makes development **organized, reusable, and easier to maintain**.

Basic Layered Architecture of Embedded Software

- Most embedded systems follow a layered architecture:
  - Application Layer
  - System / RTOS
  - Device Drivers
  - Hardware Abstraction Layer (HAL)
  - Hardware
- Each layer has a specific function.
- HAL provides a standard interface between hardware and software; purpose is portability, hardware independence, …
- Device drivers control specific hardware components such as GPIO driver, UART driver, ADC driver, SPI / I2C driver.

Middleware Layer

- Middleware provides common services used by applications; Examples -
  - Communication protocols
  - File systems
  - Networking stacks
- Typical middleware:
  - TCP/IP stack
  - USB stack
  - Bluetooth stack
- Middleware simplifies complex operations.

Application Layer

- This is the main program that performs the system's function.
- Examples of application logic:
- Temperature control
- Motor control
- Alarm monitoring
- Display management
- The application layer interacts with the RTOS and device drivers to perform tasks.

Application Layer, … cont’

- Application Layer Example in pseudocode:

```
while(1)
{
    temperature = read_sensor();
    display(temperature);

    if(temperature > limit)
        turn_on_alarm();
}
```

Summary: The **software framework of embedded systems** organizes software into components such as:

Layer	Function
Hardware	Physical devices
HAL	Hardware abstraction
Drivers	Control hardware (I/O)
RTOS	Manage tasks of a process
Middleware	Communication services
Application	Main system logic for user

Real-time operating system: Definitions

- A real time operating system (RTOS):
  - is an operating system (OS) that has system software required to synchronize and schedule the tasks in a multitasking system environment in real time, and
  - takes care of the real-time constraints in the system.
  - required when the system requires several jobs to be done at the same time and in real time space.
  - is multitasking operation system for the applications with hard or soft real time constraints.
  - is characterized by having time as a key parameter.

Real-time operating system: Definitions

- Real-time operating system (RTOS) is an operating system intended to serve real time application that process data as it comes in, mostly without buffer delay.
- A real-time operating system (RTOS) must be reliable;
- it must be fast and responsive, manage limited resources and schedule tasks so,
- they complete on time, and ensure functions are isolated and free of interference from other functions.

Real-time operating system: Characteristics

- It has better reliability;
- Its result is more predictable because its every action is executed into predefined time frame.
- Affordable Cost.
- Consume low memory.
- Kernel helps for storing the states of interrupted tasks for execution at appropriate time frame
- RTOS has better response time for highly predictability.
- Occupy less resource, … others

Real-time operating system: Functions

- The primary function of RTOS is, it provides the better management of RAM and processor as well as it gives the access to all system resources.
- Higher Priority Scheduling – helps to activate process which has high priority.
- System Clock Interrupt Routine for Time Frame – System Clock Interrupt Routine helps to perform the highly time sensitive instructions in RTOS with using system clocks.
- Deterministic Nature – provides the protection in using big length tasks
- Synchronization and Messaging - For making the communication medium in among of all tasks of one system to other system, RTOS use the synchronization and messaging.

Real-time operating system: Components

- The Scheduler: tells that in which order, the tasks can be executed.
- Symmetric Multiprocessing component (SMP): a number of multiple different tasks can be handled by the RTOS so that parallel processing can be done.
- Function Library: acts as an interface that helps you to connect kernel and application code.
- Memory Management: the most important element of the RTOS.
- Fast dispatch latency components: an interval between the termination of the task and the actual time taken by the thread, which is in the ready queue.
- User-defined data objects and classes

Real-time operating system: Applications

- Air traffic control system.
- Systems that provide immediate updating.
- Used in any system that provides up to date and minute information on stock prices.
- Defense application systems like RADAR.
- Networked Multimedia Systems
- Internet Telephony
- Heart Pacemaker , … others

Inter process communication in RTOS

- Inter-Process Communication (IPC) is a set of techniques for the exchange of data among two or more threads in one or more processes, which is similar to GPOS.
- Processes may be running on one or more computers connected by a network.
- A process is *independent* if it cannot be affected by the other processes executing in the system.
- But, a process is cooperating if it can affect or be affected by the other processes executing in the systems.
- Cooperating processes need IPC mechanism that will allow them to exchange data and information.

Inter process communication in RTOS

- Two models of IPC: **Shared memory and Message passing**
- **Shared memory:** ***without kernel interference***
  - a region of memory that is shared by cooperating processes is established,
  - processes can exchange information by reading and writing data to the shared region
- **Message passing:** ***with the interference of kernel***
  - Message Passing provides a mechanism for processes to communicate and to synchronize their actions without sharing the same address space.
  - The producer typically uses *send*() system call to send messages, and the consumer uses *receive*() system call to receive messages
  - If P and Q wish to communicate, they need to establish a communication link between them and exchange messages via send/receive.
  - send (P, message) – Q sends a message to process P

Real time task scheduling

- Task scheduling forms the basis of multitasking.
- ***Multitasking*** *is the process of scheduling and switching the CPU between several tasks.*
- A real-time system is typically developed following a concurrent programming approach in which a system may be divided into several parts, called tasks, and
- each task, which is a sequence of operations, executes in parallel with other tasks.
- A task may issue an infinite number of instances called jobs during run-time.
- Scheduling policies forms the guidelines for determining which task is to be executed when.
- The scheduling policies are implemented in an algorithm and it is run by the kernel as a service.
- The kernel service/ application, which implement the scheduling algorithm, is known as 'Scheduler.'

Scheduling accomplished when?

- The process scheduling decision may take place when a process switches its state to:
  - “Ready” state from “Running‟ state
  - “Blocked/Wait‟ state from “Running‟ state
  - “Ready‟ state from “Blocked/Wait‟ state
  - “Completed‟ state
- We call these switching the transition of process.

Terminologies in scheduling

- The selection of a scheduling criteria/algorithm should consider:
  - CPU Utilization: should always make the CPU utilization high.
  - Throughput**:** number of processes executed per unit of time; it should be higher for a good scheduler.
  - Turnaround Time**:** amount of time taken by a process for completing its execution; it should be a minimum for a good scheduling algorithm.
  - Waiting Time: time spent by a process in the 'Ready' queue waiting to get the CPU time for execution.
  - Response Time: time elapsed between the submission of a process and the first response.

Real time task scheduling, ….

- Main goal of a RTOS scheduling is to meeting deadlines.
- The scheduler in a RTOS is designed to provide a predictable (normally described as deterministic) execution pattern.
- The **scheduler**, also called the **dispatcher**, is the part of the kernel responsible for determining which task will run next.
- The scheduling is managed with the kernel.
- *In RTOS, a **thread** of execution is called a **task**.*

Cyclic Scheduling

- **Cyclic Scheduling** is a **time-driven scheduling technique** used in **real-time embedded systems**.
- Tasks are executed in a **fixed, repeating cycle.**
- Time is divided into **frames (minor cycles)**
- A set of frames forms a **major cycle**
- Tasks are assigned to specific frames i,e. each frame executes **predefined tasks**

```
Time → | Frame1 | Frame2 | Frame3 | Frame4 | → Repeat
```

Cyclic Scheduling: (Cyclic Executive)

- Cyclic scheduling required for tasks that are executing in turn, known as cyclic executive tasks.
- Each task is allowed to run to completion before it hands over to the next.
- But, a task cannot be discontinued as it runs.
- Cyclic scheduling is an important way to sequence tasks in a real-time system.
- It is static – computed offline and stored in a table. See fig. below.

$$T(t_k) = \begin{cases} T_i & \text{if } T_i \text{ is to be scheduled at time } t_k \\ I & \text{if no periodic task is scheduled at time } t_k \end{cases}$$

Cyclic Schedule Table:

- Table executes completely in one hyperperiod H; then repeats H which is least common multiple of all task periods, n quanta per hyperperiod for n a given time slice.
- **Example**: Consider a system with four tasks T1 = (1,4), T2 = (1.8, 5), T3 = (1, 20), T4 = (2, 20).

  First we determine the hyperperiod or size of major cycle based on LCM of each task's period and then select frame size.
- Thus, hyperperiod or Major cylce is LCM(4,5,20,20) ≥ 20,

  Major cycle or Hyperperiod H = 20.

Cyclic Schedule Table:, …

T1 = (1,4), T2 = (1.8, 5), T3 = (1, 20), T4 = (2, 20)

which is given as *Task = (computation-time, period)*

- Table starts out with:
  (0, T1), (1, T3), (2, T2), (3.8, I), (4, T1), …
- We have to determine frame size f so that:
  - Timing is enforced only at frame boundaries;
  - Each task is executed as a function call and must fit within a single frame;
  - Multiple tasks may be executed in a frame;
  - Number of frames per hyperperiod is F = *H/f;*

Cyclic scheduling, … Example 1, cont'

- Criteria to determine frame size:

1. Tasks must fit into frames for frame size f; then,
   - f ≥ Ci for all tasks' computation time Ci; Ci is laregest comp. time
   - Justification: Non-preemptive tasks should finish executing within a single frame.
2. Frame size f must evenly divide *hyperperiod* H:
3. There should be a complete frame between the release and deadline of every task.

Therefore, $2f - \gcd(D_i, f) \leq D_i$ for each task i; *Di is the smallest deadline with a task i*

Cyclic scheduling, … Example 1

- Based on above discussion we prepare a schedule for a system with four tasks given:

  (T1 = (1,4), T2 = (1.8, 5), T3 = (1, 20), T4 = (2, 20))

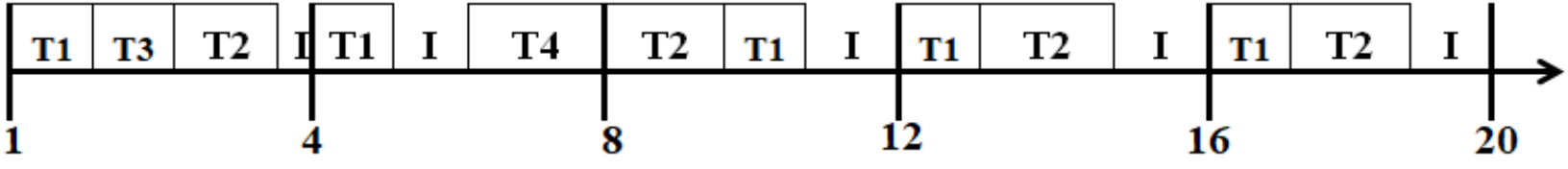
  H = LCM (4,5,20) = 20 thus, hyperperiod H=20

✓ By Constraint 1: f ≥ 2, since 2 is largest among given.

✓ By Constraint 2: f might be 1, 2, 4, 5, 10, or 20,

✓ By Constraint 3: 2 and 4 works, but larger size is preferable.

There fore frame size can be f=4 and schedule is as follows:



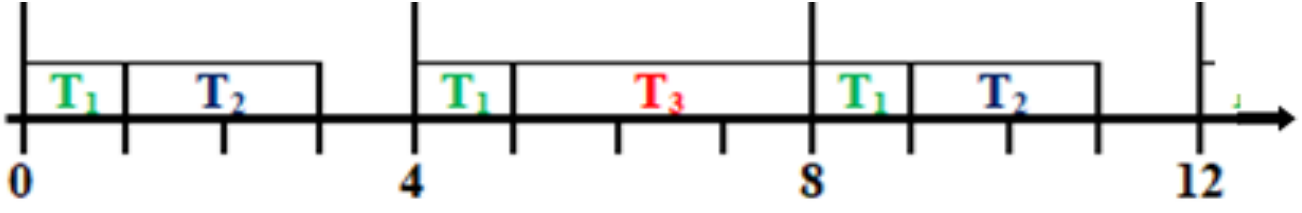
Cyclic scheduling, …

Example 2: prepare the cyclic scheduling for the following three tasks:

- T_1=(1,4) T_2=(2,6) T_3=(3,12)
- Solution: first we have to determine major cycle and minor cycle (hyperperiod H and frame size f).
  - H=LCM(4,6,12)=12
  - Frame size:
    - By 1: f≥3
    - By 2: f = {1,2, 3, 4, 6, 12}
    - By 3: f≤4 , (2f-gcd(f, Ti) ≤Di) should hold true.

Cyclic scheduling, … Example 2

- By constraint 3, i.e, (2f-gcd(f, Ti) ≤Di:
- Let f=3, (2*3 – gcd(3,4) ≤ 4 => 6-1 ≤4) is false.
- Let f=4, (2*4-gcd(4,4) ≤4 => 8-4 ≤4) is true.
- Let f=6, (2*6-gcd(6,4) ≤4 => 12-2 ≤4) is false,… same to remaining.
- Generalizing the larger value leads to false, we can stop taking f:
- f=4, the only frame size.
- Finally the cyclic scheduling table can be as follows:



Take task slicing: … when do we take?

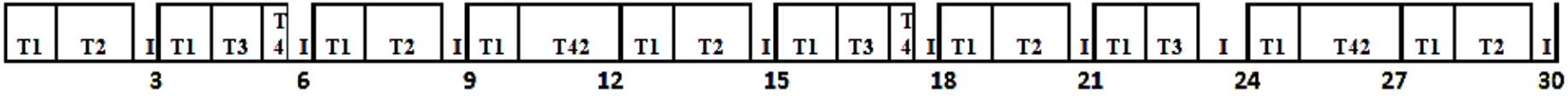
- If we cannot get the correct frame size based on constrains or the frame size constrain cannot be met, we try to take task slicing, i.e.;
  - the task with larger computational time divided into two or more sub parts without changing its period in all cases.
- Also, sometime the frame size is obtained but the tasks are not incorporated within that frame correctly, we need try to take task slicing and researching correct frame size.
- We have to know that all smaller sub-tasks we sliced must be computed within a given period only.

Example: taking task slice

- Find major cycle H and minor cycle or frame size f for the following tasks; show steps.
- Then draw the correct cyclic scheduling table based on frame size you obtained; take task slice if required.

  T_1=(1, 3) T_2=(1.5, 6) T_3=(1, 10) T_4=(2.5, 15)

- Solution: H=LCM(3,6,10,15)=30
- Frame size f: 1. f ≥ 2.5; 2. f={1,2,3,5,6,10,15,30};
- 3. 2*f-gcd(Di,f) ≤ Di; it works for f=3 only; thus, f=3.
- But as we see, T4 needs 2.5 unit which is approaching to full frame size.
- If T4 takes execution in a given frame, that impedes T1 taking its periodic execution in a frame where T4 took.
- Thus, we try to take task slicing, i.e. largest task is T4 and make T4 into two sub-tasks: T4=(0.5,15) and T42=(2,15).
- Now we can incorporate all tasks correctly in a table as shown below.



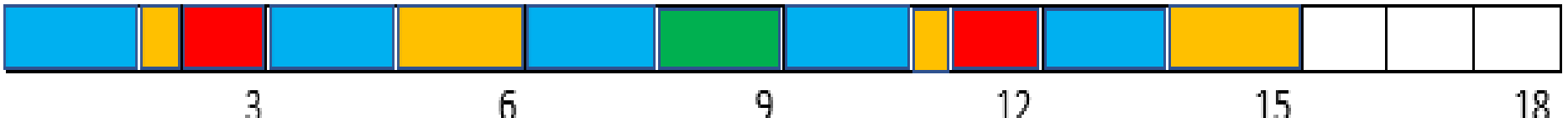
Challenges in cyclic scheduling:

- Some time scheduling table cannot be created easily, or it may not possible at all.
- Although we obtain the correct frame size, there is chance that we may not get the fittest schedule table.
- Even if task slicing may not solve these type problems too.
- Therefore, the application cannot be applicable or more digging required from analysts side, to have different analysis to that application, to be scheduled correctly.
- Unfortunately, it can be determined un-schedulable task and can be rejected too.
- See example next slide

Example: non schedulable tasks

- Find major cycle H and minor cycle or frame size f for the following
- Then draw the correct cyclic scheduling table based on frame size you obtained. T_1=(1.5, 3) T_2=(1, 6) T_3=(2, 9) T_4=(2.5, 18)

- Solution: H=LCM(3,6,9,18)=18;
- Frame size: 1. f ≥ 2; 2. f={1,2,3,6,9,18};
- 3. 2*f-gcd(Di,f) ≤ Di; it works for f=2 and f=3; so that f=3.
- But, we cannot create correct cyclic scheduling table as it will be insufficient after fifth frame i.e. after time 15, there are three task not completed their execution such as T1=1.5, T2 =1 and T4=1, but available size is only 3, no any free left inside.
- Therefore, the above tasks are taken as not schedulable; rejected

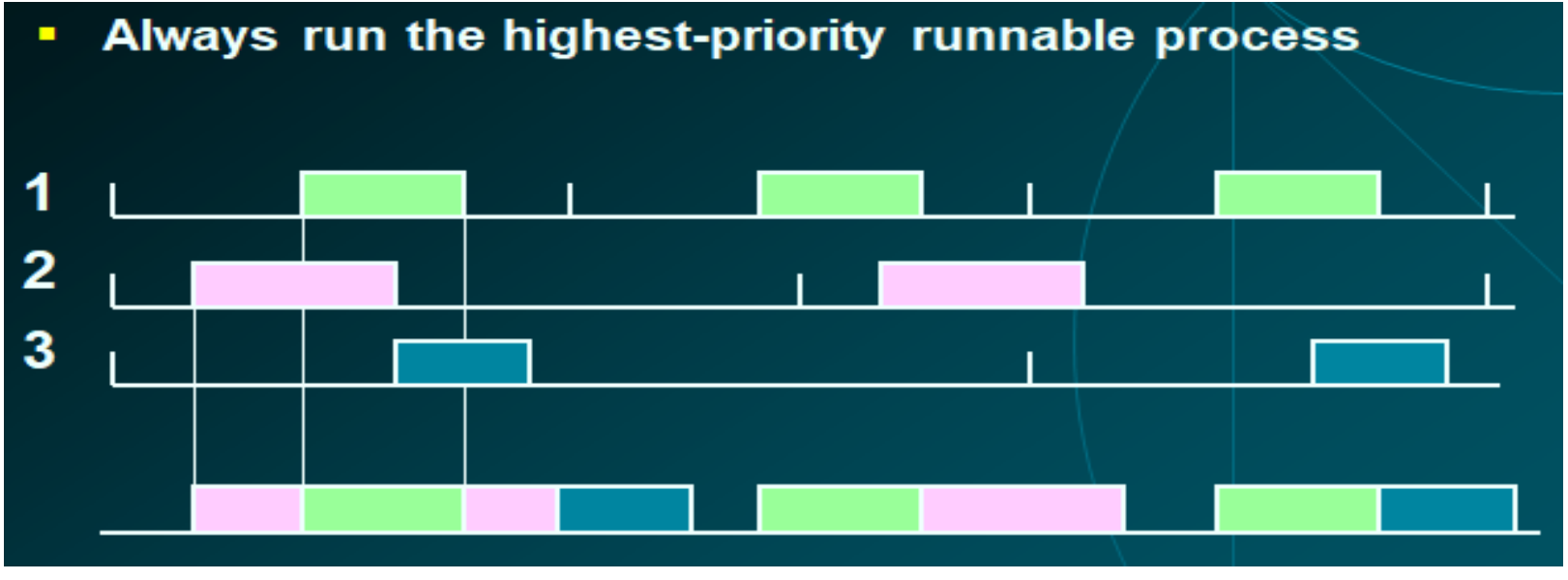


Priority-based scheduling

- Priority based scheduling enables us to give better service to certain processes in context switching.
- In multi-queue scheduling, priority was adjusted based on whether a task was more interactive or compute intensive.
- If our task is competing with other high priority tasks, it may not get as much time as it requires.
- Typical RTOS based on fixed-priority preemptive scheduler assigning each process with a priority.
- At any time, scheduler runs highest priority process ready to run to completion, unless preempted.

Priority-based scheduling

- Tasks are activated as response to events (e.g. arrival of a signal, message, etc.).
- At any given time, the highest priority ready task is continuing running. See diagram below



Multi-tasking and Concurrency issue

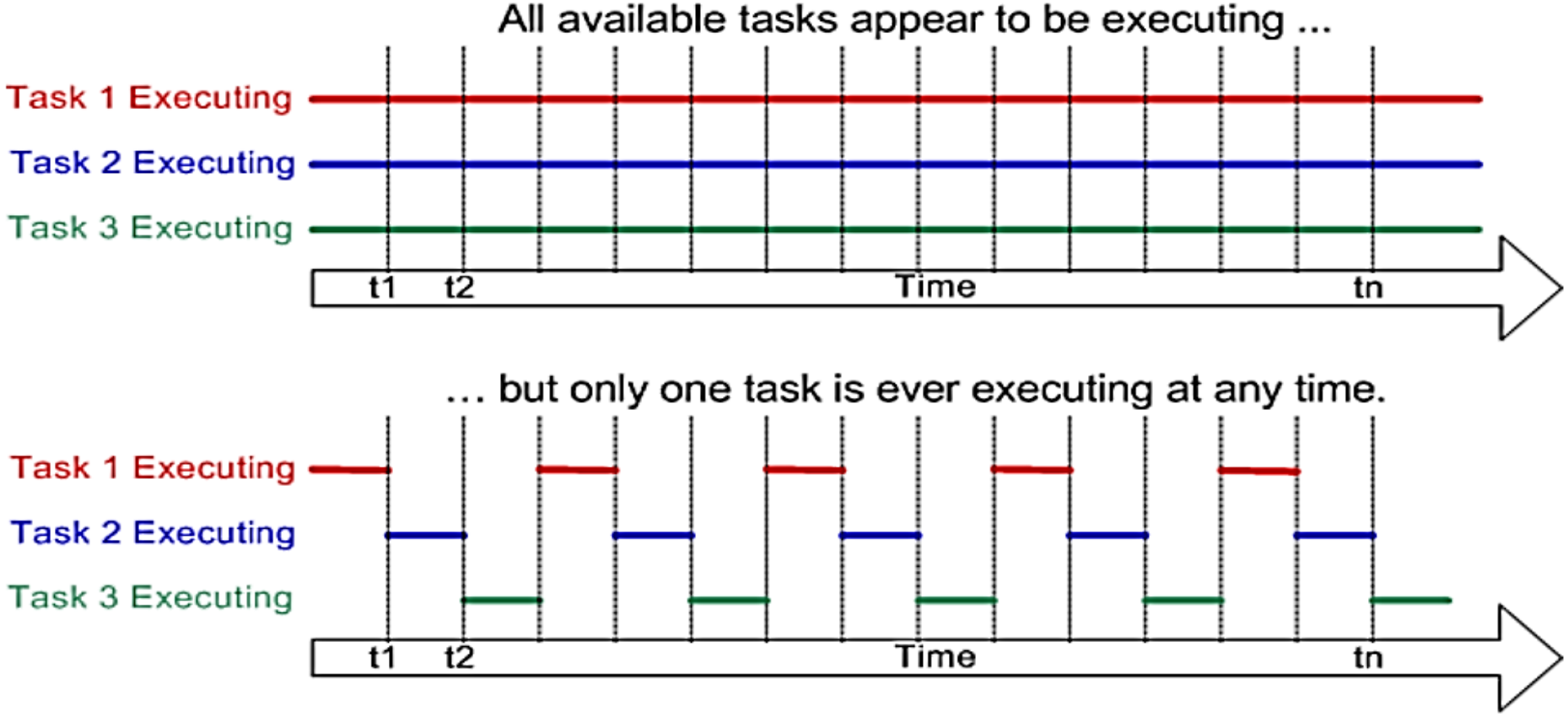
- The multitasking and inter-task communications features of the operating system allow the complex application to be partitioned into a set of smaller and more manageable tasks.
- How does the RTOS achieve multitasking?
- Each thread in an RTOS has a dedicated private context in RAM, consisting of a private stack area and a thread-control-block (TCB).
- The whole context-switch process is typically coded in CPU-specific assembly language, and takes a few microseconds to complete.

Multi-tasking and Concurrency issue

- A conventional processor can only execute a single task at a time - but
- by rapidly switching between tasks a multitasking operating system can make it appear as if each task is executing concurrently.
- This is depicted by the diagram next which shows the execution pattern of three taskswith respect to time.
- Time moves from left to right, with the colored lines showing which task is executing at any particular time.

Multi-tasking and Concurrency issue

- The upper diagram demonstrates the perceived concurrent execution pattern and the lower is the actual multitasking execution pattern.



Handling resource sharing and dependencies

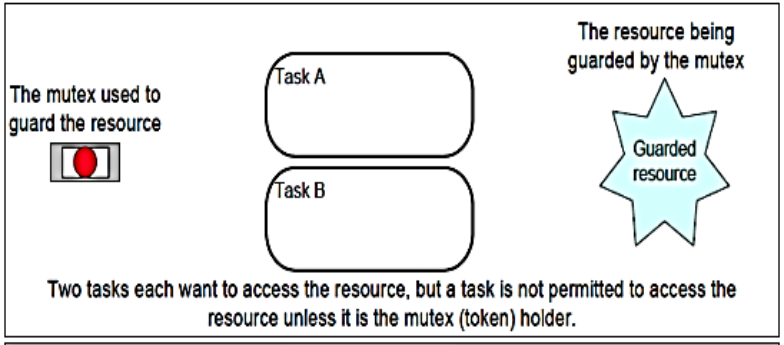
- In many applications, real-time tasks need to share some resources among themselves.
- Often these shared resources need to be used by the individual tasks in exclusive mode.
- Examples of shared resources: data structures, variables, main memory area, file, set of registers, I/O unit, etc
- Many shared resources do not allow simultaneous accesses but require mutual exclusion.
- These resources are called exclusive resources, no two threads are allowed to use at the same time.

How to protect exclusive resources?

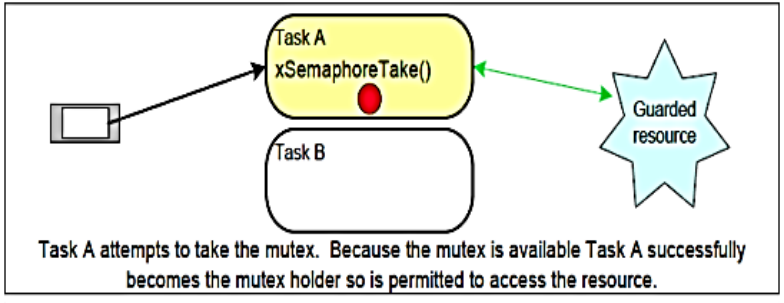
- There are several methods available to protect exclusive resources, for example
- Disabling interrupts and preemption or using concepts like semaphores that put threads into the blocked state if necessary.
- All tasks blocked on the same resource are kept in a queue associated with the semaphore.

```
...
taskENTER_CRITICAL();
   ... /* access to some exclusive resource */
taskEXIT_CRITICAL();
...
```

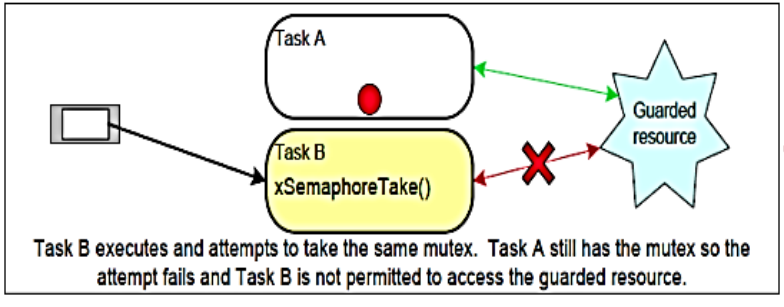
Two processes in exclusive resource



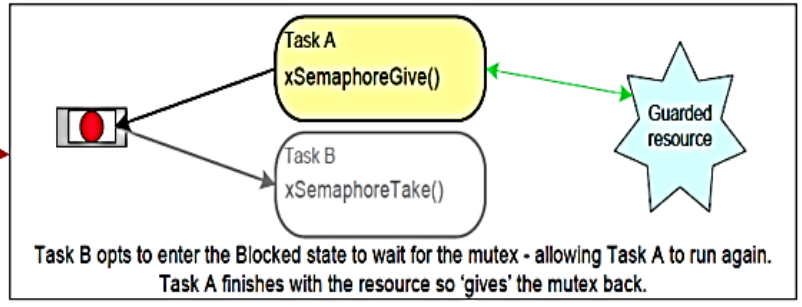
Two tasks each want to access the resource, but a task is not permitted to access the resource unless it is the mutex (token) holder.



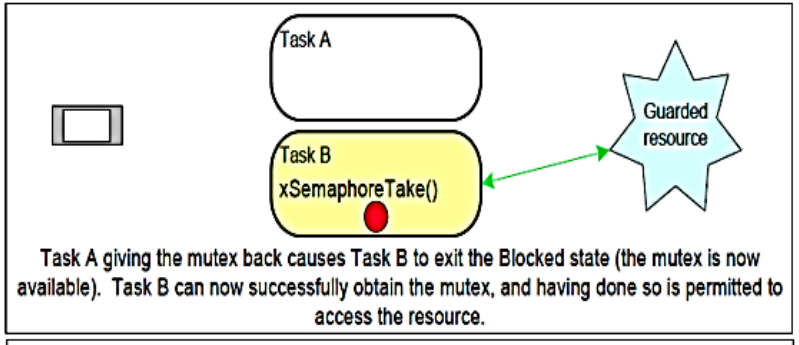
Task A attempts to take the mutex. Because the mutex is available Task A successfully becomes the mutex holder so is permitted to access the resource.



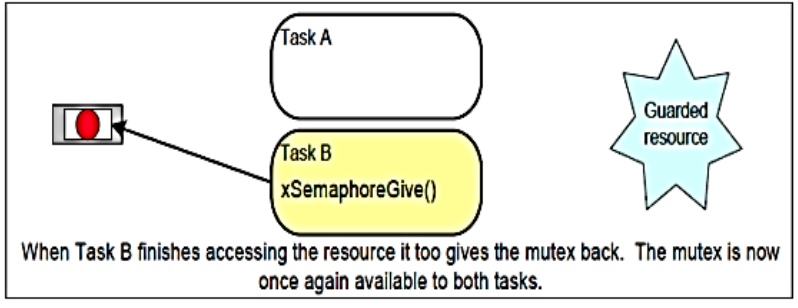
Task B executes and attempts to take the same mutex. Task A still has the mutex so the attempt fails and Task B is not permitted to access the guarded resource.



Task B opts to enter the Blocked state to wait for the mutex - allowing Task A to run again. Task A finishes with the resource so 'gives' the mutex back.



Task A giving the mutex back causes Task B to exit the Blocked state (the mutex is now available). Task B can now successfully obtain the mutex, and having done so is permitted to access the resource.



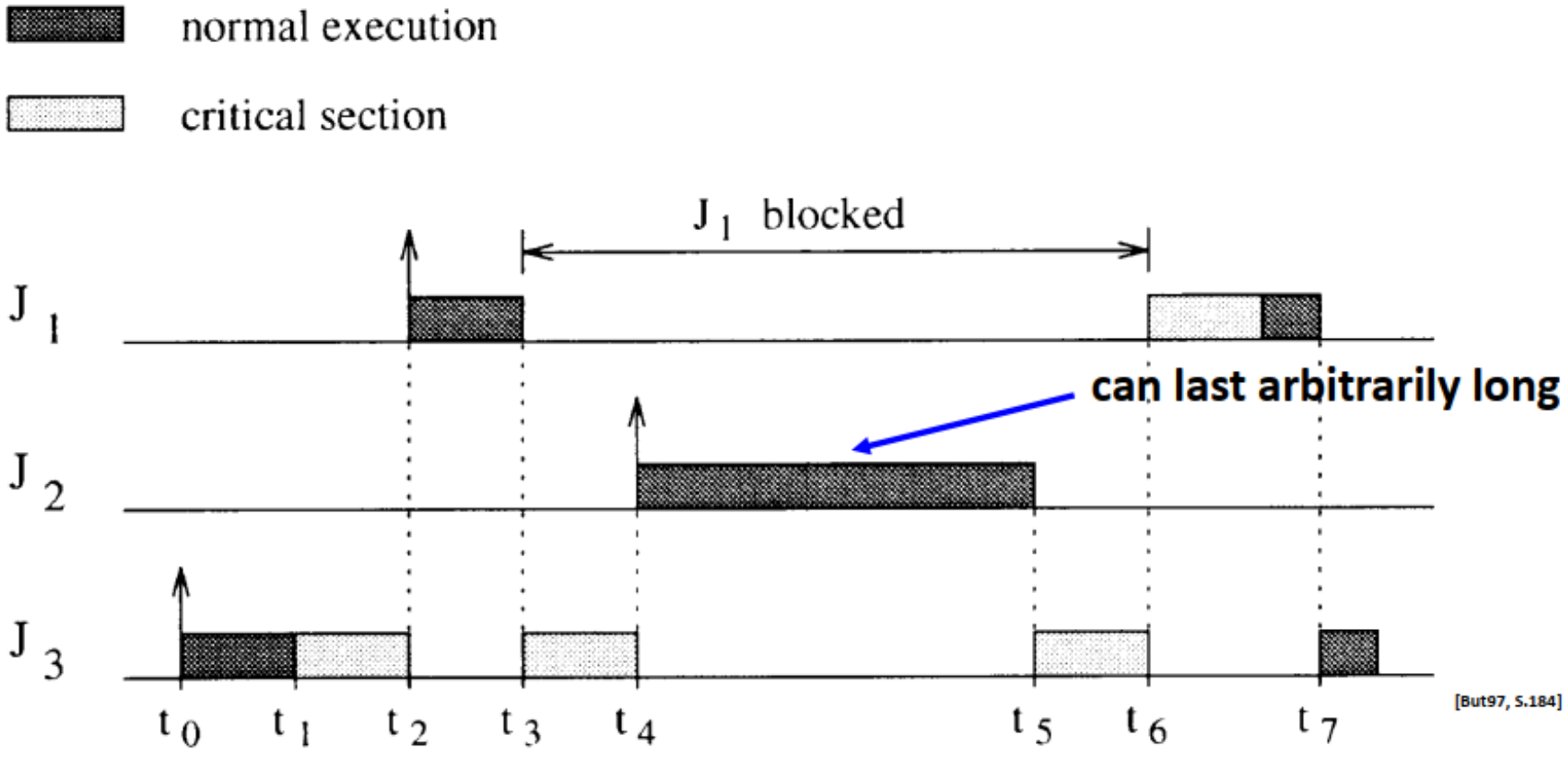
When Task B finishes accessing the resource it too gives the mutex back. The mutex is now once again available to both tasks.

Priority inversion

- The use of shared memory for implementing communication between tasks may cause priority inversion and blocking.
- Therefore, either the implementation of the shared medium is “thread safe” or the data exchange must be protected by critical sections.
- Communication always needs synchronization.
- Therefore, the timing of the communication partners is closely linked.

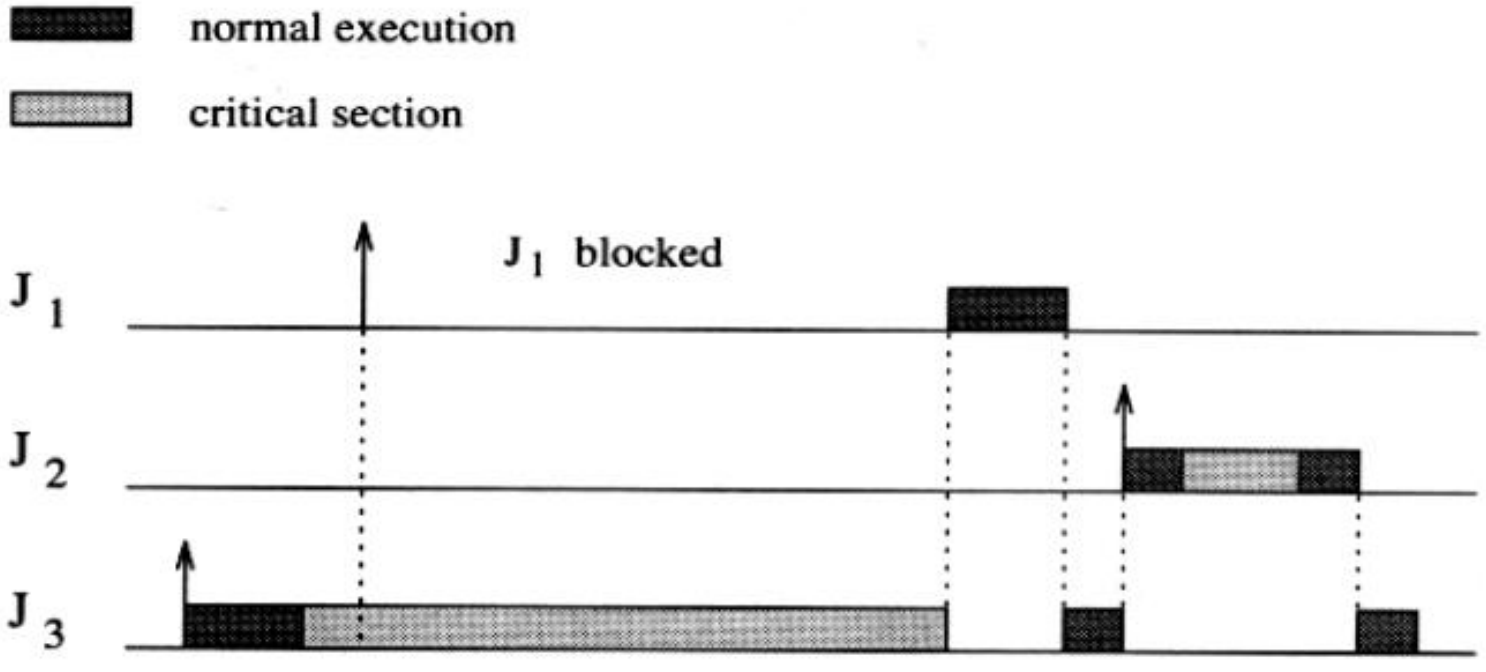
Priority inversion, … Look at diagram below

- J1, J2, J3 in priority order higher to lower



Priority inversion, …

- Disallow preemption during the execution of all critical sections.
- Simple approach, but it creates unnecessary blocking as unrelated tasks may be blocked.



Resource sharing protocols – Basic idea:

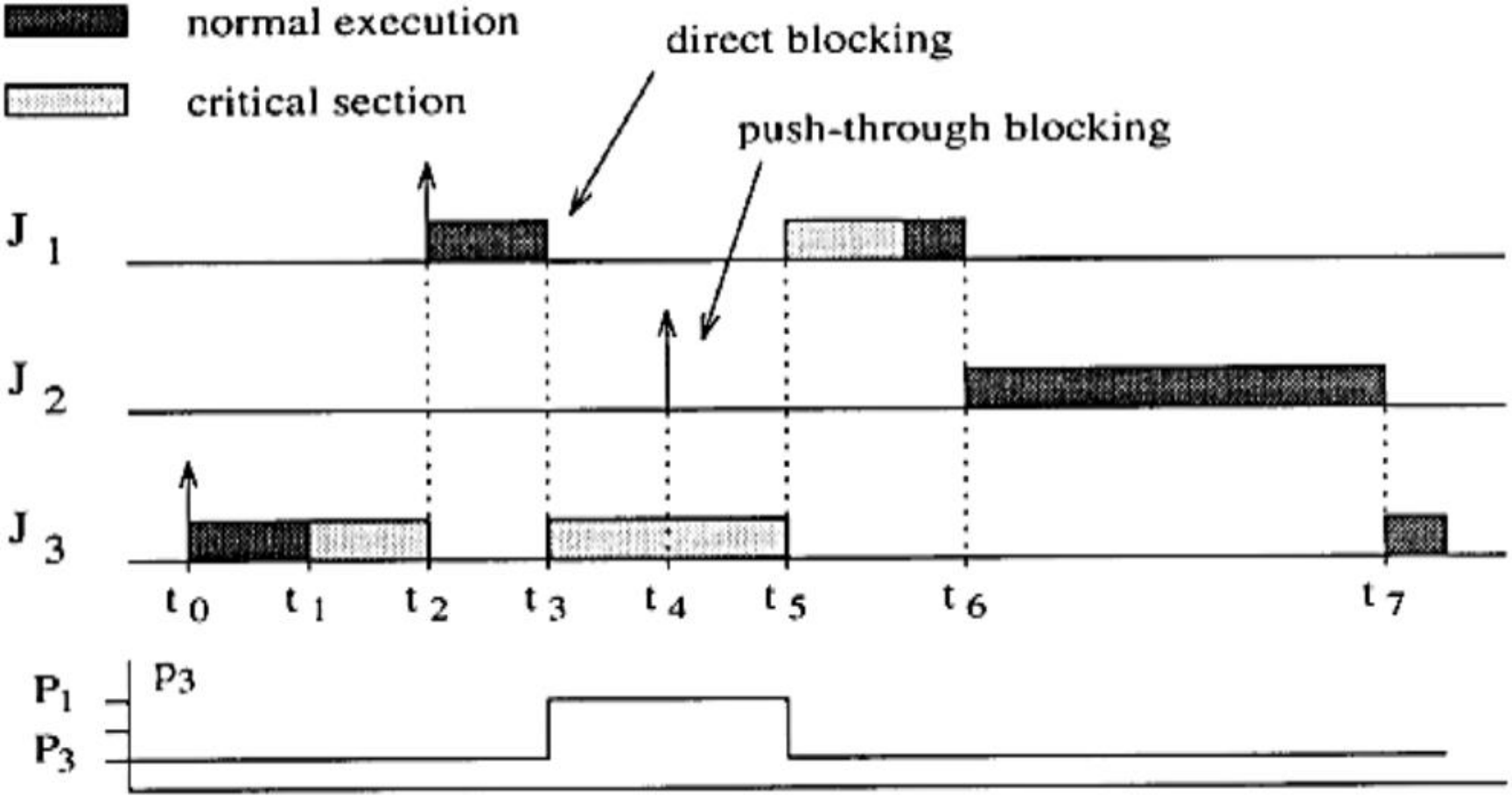
- Modify the priority of those tasks that cause blocking.
- When a task J_i blocks one or more of higher priority tasks, it temporarily accepts a higher priority chance.
- Then it temporarily assumes (inherits) the highest priority of the blocked tasks.

- ***From pdf file given, please read about Specific Methods used to handle shared resource and its access protocol.***

Resource sharing protocols – Basic idea:

- Modify the priority of those tasks that cause blocking.
- When a task J_i blocks one or more of higher priority tasks, it temporarily accepts a higher priority chance.
- **Specific Methods – shared resource access protocol:**
  - Priority Inheritance Protocol (PIP), for static priorities
  - Priority Ceiling Protocol (PCP), an extension of PIP,
  - Stack Resource Policy (SRP), for static and dynamic priorities and others …

Example: have a look at diagram below



Fault- tolerance

- **Brainstorming**:

1. What is the fault?
2. What do you think the source of fault?
3. What we mean fault tolerance?

Fault- tolerance

- Fault is an erroneous state of software or hardware resulting from failures of its components.
- Fault-Tolerance is the ability of a computer system to survive in the presence of faults.
- It aims at guaranteeing the services delivered by the system despite the presence or appearance of faults.
- Main aspect of Fault Tolerant Computing (FTC) is:
  - fault detection, fault isolation and containment, system recovery, and fault Diagnosis and Repair

Fault- tolerance, … source of fault

- The major sources of fault are listed below

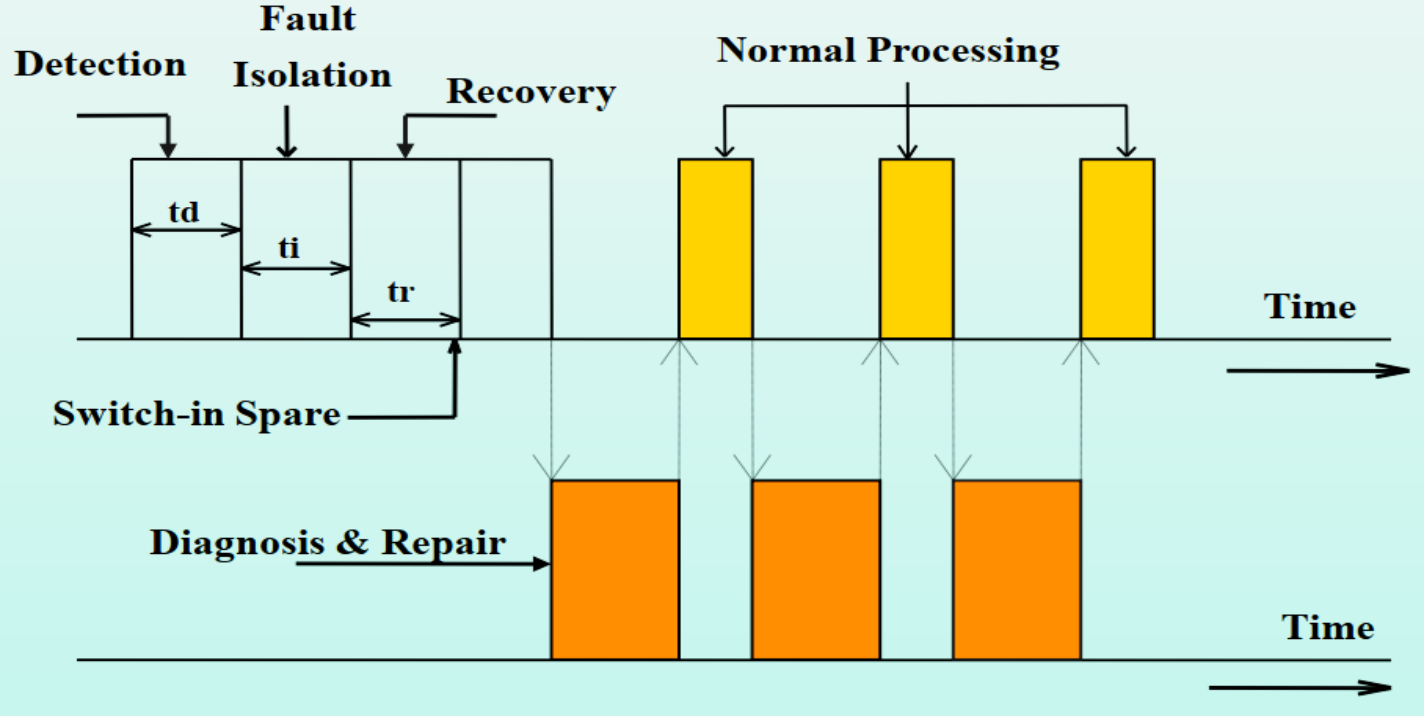
- Design errors,
- Manufacturing Problems,
- External disturbances (as harsh environmental conditions), System misuse,
- Electronic Hardware -- “bad fabrication; wears out”: latent manufacturing defects, operating environment (noise, heat, etc).
- Software -- “bad design”: Design defects and “Code rot” -- accumulated run-time faults, …. Others

Fault- tolerance, … cont’

- Main aspect of Fault Tolerant Computing (FTC) is:
  - Fault detection,
  - Fault isolation and containment or control,
  - System recovery, and
  - Fault Diagnosis and Repair.

- Look at diagram in next slide

- There are four-fold categorization or methods to deal with the system faults and increase system reliability and/or availability known as fault toleration.

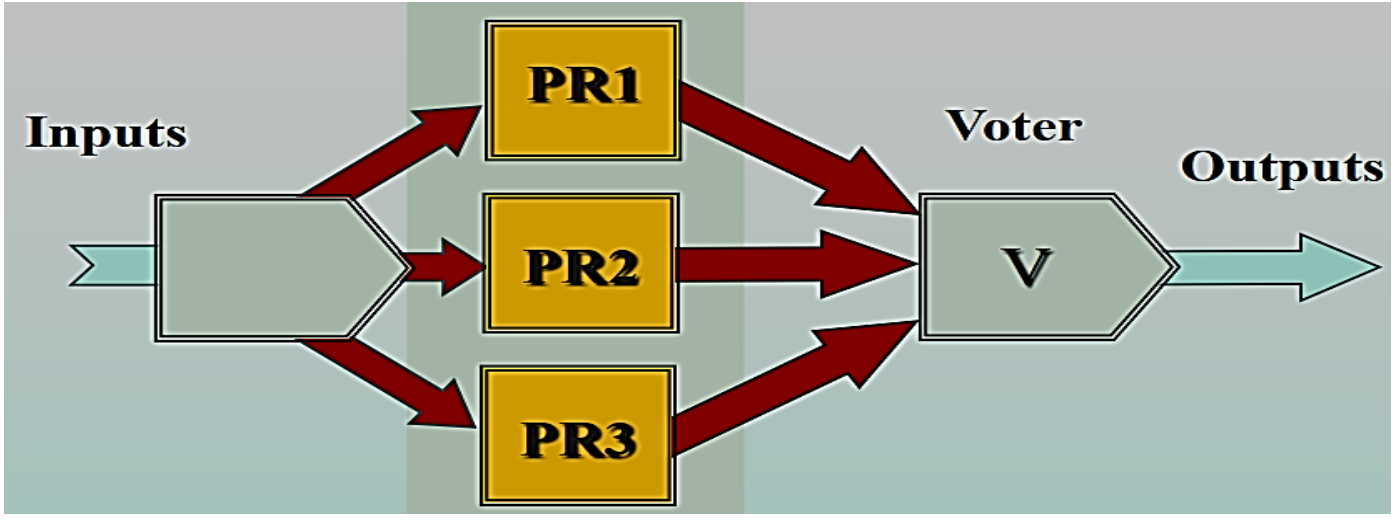


Fault – tolerance techniques

- There are two techniques such as, hardware fault tolerance and software fault tolerance.
- Hardware techniques : provides tolerance against physical i.e. hardware faults; some are:
  - Fault Detection – gives warning when a fault occurs,
  - masking Redundancy – redundant copies of an element are installed such as TMR (Triple Modular Redundancy) ,
- Software Fault-tolerance: is based on HW Fault-tolerance, it is a bigger challenge.
  - uses design redundancy to mask residual design faults of software programs
  - It can use a watchdog to figure out if the program is crashed and change the specification to provide low level of service.

Hardware Fault-Tolerance Technique by TMR:

- TMR Configuration:
  - PR1, PR2 and PR3 processors execute different versions of the code for the same application.
  - Voter compares the results and forward the majority vote of results (two out of three).



Synchronization techniques

- Clock synchronization is a method of synchronizing clock values of any two nodes in a distributed system with the use of external reference clock or internal clock value of the node.
- The requirement for communication and synchronization between tasks is very common.
- This represents a key part of the functionality provided by an RTOS known as synchronization.
- There are three broad paradigms for inter-task communications and synchronization: Task-owned facilities, Kernel objects, Message passing.

Next … detail

Synchronization techniques, …

- Task-owned facilities – attributes that an RTOS imparts to tasks that provide communication (input) facilities. For example signals.
- Kernel objects – facilities provided by the RTOS which represent stand-alone communication or synchronization facilities. Examples include: event flags, mailboxes, queues/pipes, semaphores and mutexes.
- **Message passing** – a rationalized scheme where an RTOS allows the creation of message objects, which may be sent from one task to another or to several others.
  - This is fundamental to the kernel design and leads to the description of such a product as being a “message passing RTOS”.

Clock Synchronization Algorithms

- Issues in Clock Synchronization are: each node has to send a request message "*time=?*" to the real-time server.
- The node gets a reply message with "*time=t*".
- This method has following two issues:
  - The ability of each node to read another node's clock value.
    - this can raise errors due to delay in message communication between nodes.
  - Time must never run backward since it may lead to the repetition of events or transactions creating disorder in the system.
    - Time running backward is just a perception, not actually it goes backward. *Two clock synchronization - based on above two approaches: -→*

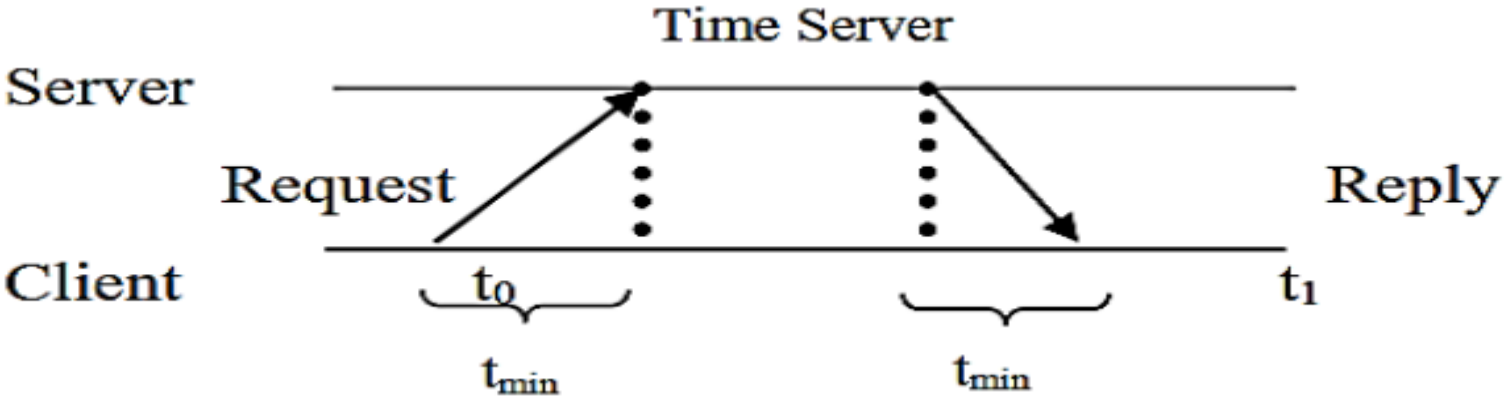
Centralized Clock Synchronization Algorithms

1. Passive Time Server:
   - Each node periodically sends a request message "time=?" to the time-server to get accurate time.
   - Time-server responds with "time=t".
   - Assume that client node send a message at time t_0 and get a reply at time t_1, then message propagation time from server to client node is (t1-t0)/2.
   - Client node receives a reply and it adjusts its clock time to t + (t1-t0)/2.
   - For a more accurate time, there is need to calculate accurate message propagation time. See next

Centralized Clock Synchronization Algorithms ,… passive time server

- There are two methods proposed to improve estimated time value.
  - i) Additional information available: Assume that to handle interrupt and to process request message sent by the client, time server takes time ti.
    - Hence, better estimated time is (t1-t0-ti)/2.
    - Therefore, clock is readjusted to **t + (t1-t0-ti)/2.**
  - **ii) No additional information available (Cristian Method):**
    - A simple estimated time t + (t1-t0)/2, is accurate if nodes are on same network.
    - For nodes on different network the minimum transmission time can be estimated as follows:
    - The time t_{min} (time needed by server to prepare reply message), and the same time t_{min} is needed by client process to dispatch reply message as shown in the figure below.

Centralized Clock Synchronization Algorithms ,… passive time server



- When the reply message arrives, the time taken by server‟s clock is in the range of (t1-t0)/2 to 2tmin.
- Therefore, accuracy of clock time is ± (t1-t0)/2 to $2t_{min}$.
- There are some limitations, these are:
  i. There is a single time server that might fail and thus synchronization temporarily unavailable.
  ii. There may be faulty time-server that reply with spurious time or an imposter time server that replied with incorrect time.

Centralized Clock Synchronization Algorithms, …

2. Active Time Server:

- Time server periodically broadcasts its clock time as "time=t".
- Other nodes receive message and readjust their local clock accordingly.
- Each node assumes message propagation time = ta, and readjust clock time = t + ta.
- There are some limitations such as:
  i. Due to communication link failure, message may be delayed and clock readjusted to incorrect time.
  ii. Network should support broadcast facility.

Centralized Clock Synchronization Algorithms, …

3. Berkeley Algorithm:

- This algorithm overcomes limitation of faulty clock and malicious interference in passive time server and also overcomes limitation of active time server algorithm.
- Time server periodically sends a request message "time=?" to all nodes in the system.
- Each node sends back its time value to the time server.
- Time server takes an average of other computer clock's value including its own clock value and readjusts its own clock accordingly.
- It avoids reading from unreliable clocks.
- For readjustment, time server sends the factor by which other nodes require adjustment.
- The readjustment value can either be +ve or –ve.
- It has also the limitations *such as*:
  - due to centralized system single point of failure may occur.
  - A single time-server may not be capable of serving all time requests from scalability point of view.

Distributed Clock Synchronization Algorithm

- Distributed clock synchronization algorithm overcomes issue of scalability and single point of failure as there is no common or global clock required.
- Processes make decisions based on local information and relevant information distributed across machines.

Global Averaging Algorithm: Each node in the system broadcast its local clock time in the form of special “resync“ message when its local time is $t_0 + IR$, where I is for interrupt time and R includes number of nodes in the system, maximum implication rate etc. *Detail,… pdf*

Distributed Clock Synchronization Algorithm

- After broadcasting, clock process of the node waits for some time period t.
- During this period, it collects „resync" message from other nodes and records its receiving time based on its local clock time.
- At the end of waiting period, it estimates the skew of its own clock with respect to all other nodes.
- To find the correct time, need to estimate skew value. *(read skew procedure from pdf)*

Distributed Clock Synchronization Algorithm

Localized Averaging Algorithm:

- This algorithm overcomes limitation of scalability of global averaging algorithm.
- Nodes of the system are arranged logically in a specific pattern like a ring or grid.
- Periodically each node exchange its local clock time with its neighbors and then sets its clock time to the average of its own clock time and the clock time of its neighbors.

RTOS support for semaphores, queues, and events

- *Read content detail from pdf file given*

Any

Chapter 4 … Next